PROJECT CODE:

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <SPI.h>
#include <MFRC522.h>
#include <LiquidCrystal_I2C.h>
#include <ESP32Servo.h>
#include <UniversalTelegramBot.h>
#include "time.h"

// --- NETWORK CONFIG ---
const char* ssid = "OPPOA3Pro5G";
const char* password = "123321123";
#define BOTtoken "8229452516:AAEWF1eAmIsPWSyLiCcXeOQktRA5o1AKpI8"
// first part is the BOT ID
//second one Authentication Token

// --- NTP TIME CONFIG ---
const char* ntpServer = "pool.ntp.org"; // address of the clock
const long  gmtOffset_sec = 19800; // IST Offset (India)
const int   daylightOffset_sec = 0;

// --- PIN DEFINITIONS ---
#define SS_PIN    5
#define RST_PIN   4
#define SERVO_PIN 14
#define IR_PIN    2   // Close gate sensor
#define GREEN_LED 13
#define RED_LED   12
#define TRIG_PIN  15  // Ultrasonic Trigger
#define ECHO_PIN  27  // Ultrasonic Echo
```

```
struct UserData {
  byte uid[4];
  String name;
  String chatID;
  bool isInside;
};

UserData users[] = {
  {{0x7D, 0xBD, 0x70, 0x06}, "Ashrith Sai", "6373138860", false},
  {{0xD6, 0xF9, 0x50, 0x06}, "Arun Kumar",  "6373138860", false}
};
const int TOTAL_USERS = sizeof(users) / sizeof(users[0]);

MFRC522 rfid(SS_PIN, RST_PIN);//object RFID reader
LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo gateServo;//object for servo motor
WiFiClientSecure client;//secure internet connection
UniversalTelegramBot bot(BOTtoken, client);

int totalLogs = 0;//counter
const int legThreshold = 18;//in cm // sensitivity of Ultrasonic sensor

void setup() {
  Serial.begin(115200);// fast &11520 bytes per second & waitime low

  // 1. Hardware Init
  lcd.init();
  lcd.backlight();
  lcd.print("System Booting");
```

```
pinMode(IR_PIN, INPUT);
pinMode(GREEN_LED, OUTPUT);
pinMode(RED_LED, OUTPUT);
pinMode(TRIG_PIN, OUTPUT);
pinMode(ECHO_PIN, INPUT);

ESP32PWM::allocateTimer(0);//uses timer-0
gateServo.setPeriodHertz(50);
gateServo.attach(SERVO_PIN, 500, 2400);//connects the s/w with h/w
gateServo.write(55); // Starting Closed Position

// 2. WiFi Connection
lcd.clear();
lcd.print("WiFi Connecting");
WiFi.begin(ssid, password);

while (WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");
  digitalWrite(RED_LED, !digitalRead(RED_LED));
}

digitalWrite(RED_LED, LOW);
Serial.println("\nWiFi Connected!");

// 3. Time Sync
client.setInsecure();
configTime(gmtOffset_sec, daylightOffset_sec, "time.google.com", ntpServer);

//wait room
lcd.clear();
```

```
  lcd.print("Syncing Time...");
  struct tm timeinfo;
  while (!getLocalTime(&timeinfo)) {
    delay(500);
    Serial.println("Waiting for NTP...");
  }

  // 4. Initialize RFID
  SPI.begin();
  rfid.PCD_Init();//wake up rfid reader
  printTableHeader();
  lcd.clear();
  showReadyScreen();
}

void loop() {
  if (rfid.PICC_IsNewCardPresent() && rfid.PICC_ReadCardSerial()) {
    int idx = findUser(rfid.uid.uidByte);
    if (idx != -1) {
      if (users[idx].isInside){
        logToSerial(idx, "RE-ENTRY DENY");
        handleReEntry(idx);
      } else {
        processSecureEntry(idx);
      }
    } else {
      logToSerial(-1, "UNKNOWN CARD");
      denyUnknown();
    }
    rfid.PICC_HaltA();//sleep mode
    rfid.PCD_StopCrypto1();//stops the comm
```

```
  }
}

// --- CORE LOGIC FUNCTIONS ---

void processSecureEntry(int idx) {
  int legCount = 0;
  bool objectDetected = false;
  unsigned long scanTime = millis();
  int lastSec = -1;

  lcd.clear();
  lcd.print("SCANNING LEGS");

  while (millis() - scanTime < 3500) { // Increased to 3.5s for a natural walking pace
    long d = getDistance();
    int secondsLeft = 3 - ((millis() - scanTime) / 1000);

    if (secondsLeft != lastSec && secondsLeft >= 0) {
      lcd.setCursor(0, 1);
      lcd.print("Time Left: "); lcd.print(secondsLeft); lcd.print("s ");
      lastSec = secondsLeft;
    }

    // Logic: If distance drops below threshold, we found a leg
    if (d > 2 && d < legThreshold && !objectDetected) {
      legCount++;
      objectDetected = true;
      digitalWrite(GREEN_LED, HIGH); delay(50); digitalWrite(GREEN_LED, LOW);
    }
    // Wait for the person to step past the sensor before allowing another "leg" count
```

```
  else if (d >= (legThreshold + 4)) {
    objectDetected = false;
  }
  delay(30);
 }

 // --- REVISED LEG COUNT LOGIC ---
 if (legCount > 2) {
  // Multiple people (Tailgating)
  logToSerial(idx, "TAILGATE BLKD");
  lcd.clear();
  lcd.print("TAILGATE ALERT!");
  lcd.setCursor(0, 1);
  lcd.print("Legs: "); lcd.print(legCount);

  digitalWrite(RED_LED, HIGH);
  if(WiFi.status() == WL_CONNECTED) {
    bot.sendMessage(users[idx].chatID, " 🚨 Tailgate attempt blocked for: " + users[idx].name + "
(Legs detected: " + String(legCount) + ")", "");
  }
  delay(3000);
  digitalWrite(RED_LED, LOW);
 }
 else if (legCount == 2) {
  // Standard human (2 legs)
  logToSerial(idx, "ACCESS GRANTED");
  grantAccess(idx);
 }
 else if (legCount == 1) {
  // Single leg detected (incomplete movement or error)
  logToSerial(idx, "INCOMPLETE SCAN");
```

```
    lcd.clear();
    lcd.print("Only 1 Leg Seen");
    lcd.setCursor(0,1);
    lcd.print("Walk Fully Past");
    digitalWrite(RED_LED, HIGH); delay(2000); digitalWrite(RED_LED, LOW);
  }
  else {
    logToSerial(idx, "NO LEG DETECT");
    lcd.clear();
    lcd.print("No Person Found");
    delay(1500);
  }
  showReadyScreen();
}
//execution part
void grantAccess(int idx) {
  users[idx].isInside = true;
  lcd.clear();
  lcd.print("Access Granted");
  lcd.setCursor(0, 1);
  lcd.print(users[idx].name);

  digitalWrite(GREEN_LED, HIGH);
  gateServo.write(175); // Open Position

  unsigned long timeout = millis();
  bool personInGate = false;

  while (millis() - timeout < 8000) {
    if (digitalRead(IR_PIN) == LOW) {
      personInGate = true;
```

```
    }
    if (personInGate && digitalRead(IR_PIN) == HIGH) {
      delay(500);
      break;
    }
    delay(10);
  }

  gateServo.write(55); // Close Position
  digitalWrite(GREEN_LED, LOW);
}

// --- HELPER FUNCTIONS ---

String getTodayDate() {
  struct tm timeinfo;
  if(!getLocalTime(&timeinfo)) return "00/00/2026";
  char dateBuffer[12];
  strftime(dateBuffer, sizeof(dateBuffer), "%d/%m/%Y", &timeinfo);
  return String(dateBuffer);
}
//ultrasonic distance msrmt
long getDistance() {
  digitalWrite(TRIG_PIN, LOW);
   delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  long duration = pulseIn(ECHO_PIN, HIGH, 25000);
  return (duration == 0) ? 400 : (duration * 0.034 / 2);
}
```

```
int findUser(byte* scannedUID) {
  for (int i = 0; i < TOTAL_USERS; i++) {
    if (memcmp(scannedUID, users[i].uid, 4) == 0) return i;
  }
  return -1;
}

void printTableHeader() {
  Serial.println("\n+------+------------+------------+--------------------+----------------+");
  Serial.println("| ID   |    DATE    | TIME (sec) |      USER NAME     |    STATUS      |");
  Serial.println("+------+------------+------------+--------------------+----------------+");
}

void logToSerial(int idx, String status) {
  totalLogs++;
  float timestamp = millis() / 1000.0;
  String todayDate = getTodayDate();
  String name = (idx == -1) ? "Unknown User" : users[idx].name;

  Serial.print("| "); Serial.print(totalLogs);
  if (totalLogs < 10) Serial.print("    | "); else Serial.print("   | ");
  Serial.print(todayDate); Serial.print(" | ");
  Serial.print(timestamp, 1);
  if (timestamp < 100) Serial.print("        | "); else Serial.print("       | ");

  Serial.print(name);
  for (int i = 0; i < (18 - name.length()); i++) Serial.print(" ");
  Serial.print(" | ");

  Serial.print(status);
```

```
  for (int i = 0; i < (14 - status.length()); i++) Serial.print(" ");
  Serial.println(" |");
}

void handleReEntry(int idx) {
  lcd.clear();
  lcd.print("ALREADY INSIDE");
  for (int i = 0; i < 6; i++) {
    digitalWrite(RED_LED, HIGH);
     delay(500);
    digitalWrite(RED_LED, LOW);
     delay(500);
  }
  showReadyScreen();
}

void denyUnknown() {
  lcd.clear();
  lcd.print("INVALID CARD");
  digitalWrite(RED_LED, HIGH);
   delay(2000);
   digitalWrite(RED_LED, LOW);
  showReadyScreen();
}

void showReadyScreen() {
  lcd.clear();
  lcd.print("Scan RFID Tag");
}
```